

AI Support SaaS — API Endpoint Map

FastAPI multi-tenant AI customer-support platform · ~150 endpoints

This is essentially Intercom with an AI agent layer bolted on, so the real endpoint count is bigger than a first sketch once you account for the customer-facing widget half, RBAC actions, and the pieces that make it commercially viable. Grouped below to match phases 1-9 of the project first, then extended with the production-grade extras a sellable multi-tenant SaaS eventually needs.

Core — matches phases 1-9

Auth

POST	/auth/register	create account (bootstraps first org if needed)
POST	/auth/login	returns access + refresh JWT
POST	/auth/refresh	exchange refresh token for new access token
POST	/auth/logout	revoke refresh token
POST	/auth/forgot-password	send reset email
POST	/auth/reset-password	reset with token
POST	/auth/verify-email	verify email with token
POST	/auth/resend-verification	resend verification email
POST	/auth/change-password	change password (authenticated)
POST	/auth/mfa/enable	enable 2FA
POST	/auth/mfa/verify	confirm 2FA code
POST	/auth/mfa/disable	disable 2FA

Users

GET	/users/me	current user profile
PATCH	/users/me	update profile
DELETE	/users/me	delete account
POST	/users/me/avatar	upload avatar
GET	/users/me/organizations	orgs this user belongs to

Organizations

POST	/organizations	create org
GET	/organizations	list orgs for current user
GET	/organizations/{org_id}	get org
PATCH	/organizations/{org_id}	update org
DELETE	/organizations/{org_id}	delete org
POST	/organizations/{org_id}/logo	upload branding/logo
GET	/organizations/{org_id}/settings	get settings
PATCH	/organizations/{org_id}/settings	update settings
POST	/organizations/{org_id}/transfer-ownership	reassign owner
POST	/organizations/{org_id}/switch	switch active org, re-scoped JWT

Members & invitations

GET	/organizations/{org_id}/members	list members
PATCH	/organizations/{org_id}/members/{member_id}/role	change role
POST	/organizations/{org_id}/members/{member_id}/deactivate	
POST	/organizations/{org_id}/members/{member_id}/reactivate	
DELETE	/organizations/{org_id}/members/{member_id}	remove member
POST	/organizations/{org_id}/invitations	invite by email
GET	/organizations/{org_id}/invitations	list pending invites
DELETE	/organizations/{org_id}/invitations/{invitation_id}	revoke invite
POST	/invitations/{token}/accept	accept (token = auth)
POST	/invitations/{token}/decline	decline

Agents

POST	/organizations/{org_id}/agents	create
GET	/organizations/{org_id}/agents	list
GET	/organizations/{org_id}/agents/{agent_id}	get
PATCH	/organizations/{org_id}/agents/{agent_id}	update (name, prompt, model, temp...)
DELETE	/organizations/{org_id}/agents/{agent_id}	delete
POST	/organizations/{org_id}/agents/{agent_id}/publish	go live
POST	/organizations/{org_id}/agents/{agent_id}/unpublish	pause
POST	/organizations/{org_id}/agents/{agent_id}/duplicate	clone
POST	/organizations/{org_id}/agents/{agent_id}/test	sandbox test chat
GET	/organizations/{org_id}/agents/{agent_id}/versions	prompt version history
POST	/organizations/{org_id}/agents/{agent_id}/versions/{version_id}/restore	
POST	/organizations/{org_id}/agents/{agent_id}/knowledge-bases	attach a KB
DELETE	/organizations/{org_id}/agents/{agent_id}/knowledge-bases/{kb_id}	
POST	/organizations/{org_id}/agents/{agent_id}/tools	attach a tool
DELETE	/organizations/{org_id}/agents/{agent_id}/tools/{tool_id}	

Tools — function-calling registry (your Tool Call entity implies agents call things)

POST	/organizations/{org_id}/tools	define a callable tool
GET	/organizations/{org_id}/tools	list
GET	/organizations/{org_id}/tools/{tool_id}	get
PATCH	/organizations/{org_id}/tools/{tool_id}	update
DELETE	/organizations/{org_id}/tools/{tool_id}	delete
POST	/organizations/{org_id}/tools/{tool_id}/test	dry-run execution

Knowledge base & documents

POST	/organizations/{org_id}/knowledge-bases	create KB
GET	/organizations/{org_id}/knowledge-bases	list
GET	/organizations/{org_id}/knowledge-bases/{kb_id}	get
PATCH	/organizations/{org_id}/knowledge-bases/{kb_id}	update
DELETE	/organizations/{org_id}/knowledge-bases/{kb_id}	delete
POST	/organizations/{org_id}/knowledge-bases/{kb_id}/documents/upload-url	presigned S3 URL
POST	/organizations/{org_id}/knowledge-bases/{kb_id}/documents	register/upload doc
GET	/organizations/{org_id}/knowledge-bases/{kb_id}/documents	list docs
GET	/organizations/{org_id}/knowledge-bases/{kb_id}/documents/{doc_id}	get doc
DELETE	/organizations/{org_id}/knowledge-bases/{kb_id}/documents/{doc_id}	delete doc
GET	/organizations/{org_id}/knowledge-bases/{kb_id}/documents/{doc_id}/status	chunk/embed progress
POST	/organizations/{org_id}/knowledge-bases/{kb_id}/documents/{doc_id}/reindex	re-chunk + re-embed
GET	/organizations/{org_id}/knowledge-bases/{kb_id}/documents/{doc_id}/chunks	inspect chunks
POST	/organizations/{org_id}/knowledge-bases/{kb_id}/search	test retrieval
GET	/organizations/{org_id}/knowledge-bases/{kb_id}/stats	doc count, size, index health

Widget config — org side

GET	/organizations/{org_id}/widget/settings	colors, greeting, position
PATCH	/organizations/{org_id}/widget/settings	
GET	/organizations/{org_id}/widget/embed-code	the <script> snippet
POST	/organizations/{org_id}/widget/domains	allow-list a domain
DELETE	/organizations/{org_id}/widget/domains/{domain_id}	

Public widget API — customer side, phase 6/7

POST	/public/widget/{public_key}/session	start anonymous session
POST	/public/widget/{public_key}/conversations	create conversation
GET	/public/widget/{public_key}/conversations/{conv_id}/messages	history
POST	/public/widget/{public_key}/conversations/{conv_id}/messages	send message
GET	/public/widget/{public_key}/conversations/{conv_id}/messages/stream	SSE stream of AI reply

Conversations — staff/dashboard side

POST	/organizations/{org_id}/conversations	staff-initiated conversation
GET	/organizations/{org_id}/conversations	list (filter by status, assignee)
GET	/organizations/{org_id}/conversations/{conv_id}	get + metadata
PATCH	/organizations/{org_id}/conversations/{conv_id}	update status/tags
POST	/organizations/{org_id}/conversations/{conv_id}/assign	assign to human
POST	/organizations/{org_id}/conversations/{conv_id}/unassign	
POST	/organizations/{org_id}/conversations/{conv_id}/archive	
POST	/organizations/{org_id}/conversations/{conv_id}/close	
POST	/organizations/{org_id}/conversations/{conv_id}/reopen	
GET	/organizations/{org_id}/conversations/{conv_id}/notes	internal staff notes
POST	/organizations/{org_id}/conversations/{conv_id}/notes	
GET	/organizations/{org_id}/conversations/export	CSV export

Messages

POST	/organizations/{org_id}/conversations/{conv_id}/messages	send (staff reply)
GET	/organizations/{org_id}/conversations/{conv_id}/messages	list
GET	/organizations/{org_id}/conversations/{conv_id}/messages/stream	SSE token stream
POST	/organizations/{org_id}/conversations/{conv_id}/messages/{msg_id}/retry	regenerate
POST	/organizations/{org_id}/conversations/{conv_id}/messages/{msg_id}/feedback	thumbs up/down
GET	/organizations/{org_id}/conversations/{conv_id}/messages/{msg_id}/tool-calls	

A real "chat with us" widget wants push, not just poll — you'll likely want a `WS /organizations/{org_id}/conversations/{conv_id}/live` channel for the dashboard (and an equivalent for the widget) alongside the SSE stream above, so new messages and typing indicators land instantly on both sides.

Tickets

POST	/organizations/{org_id}/tickets	create (usually auto-created on handoff)
GET	/organizations/{org_id}/tickets	list
GET	/organizations/{org_id}/tickets/{ticket_id}	get
PATCH	/organizations/{org_id}/tickets/{ticket_id}	update
POST	/organizations/{org_id}/tickets/{ticket_id}/assign	
POST	/organizations/{org_id}/tickets/{ticket_id}/resolve	
POST	/organizations/{org_id}/tickets/{ticket_id}/escalate	
GET	/organizations/{org_id}/tickets/{ticket_id}/comments	
POST	/organizations/{org_id}/tickets/{ticket_id}/comments	

Analytics

GET	/organizations/{org_id}/analytics/dashboard	today's conversations, avg response time, CSAT
GET	/organizations/{org_id}/analytics/usage/daily	
GET	/organizations/{org_id}/analytics/costs	by agent / model
GET	/organizations/{org_id}/analytics/tokens	
GET	/organizations/{org_id}/analytics/conversations	volume, duration
GET	/organizations/{org_id}/analytics/agents/{agent_id}/performance	
GET	/organizations/{org_id}/analytics/top-questions	
GET	/organizations/{org_id}/analytics/csat	
GET	/organizations/{org_id}/analytics/escalations	
GET	/organizations/{org_id}/analytics/export	

Beyond the sketch — what a sellable SaaS eventually needs

API keys — server-to-server access, plus the widget's public key

GET	/organizations/{org_id}/api-keys	
POST	/organizations/{org_id}/api-keys	
DELETE	/organizations/{org_id}/api-keys/{key_id}	
POST	/organizations/{org_id}/api-keys/{key_id}/rotate	

Webhooks (outbound) — notify the org's own systems on events

GET	/organizations/{org_id}/webhooks	
POST	/organizations/{org_id}/webhooks	
PATCH	/organizations/{org_id}/webhooks/{webhook_id}	
DELETE	/organizations/{org_id}/webhooks/{webhook_id}	
POST	/organizations/{org_id}/webhooks/{webhook_id}/test	
GET	/organizations/{org_id}/webhooks/{webhook_id}/deliveries	

Billing & subscription — someone has to pay for the OpenAI calls

GET	/organizations/{org_id}/billing/subscription	
POST	/organizations/{org_id}/billing/subscription	subscribe / change plan
DELETE	/organizations/{org_id}/billing/subscription	cancel
GET	/organizations/{org_id}/billing/invoices	
GET	/organizations/{org_id}/billing/payment-methods	
POST	/organizations/{org_id}/billing/payment-methods	
DELETE	/organizations/{org_id}/billing/payment-methods/{pm_id}	
GET	/organizations/{org_id}/billing/usage	against plan limits
POST	/webhooks/stripe	inbound Stripe events

Notifications

GET	/users/me/notifications	
PATCH	/users/me/notifications/{id}/read	
PATCH	/users/me/notifications/read-all	
GET	/users/me/notification-preferences	
PATCH	/users/me/notification-preferences	

Platform admin — internal ops view across every tenant

```
GET    /admin/organizations           list every org
PATCH /admin/organizations/{org_id}/suspend
PATCH /admin/organizations/{org_id}/reactivate
GET    /admin/users
GET    /admin/analytics/platform      revenue/usage across all tenants
```

System

```
GET    /health
GET    /health/ready                 DB + vector store reachable
GET    /version
```

A few conventions baked into the above

- Everything sits behind `/api/v1/...` — omitted above for readability.
- Org-owned resources nest fully under `/organizations/{org_id}/...`; only pre-auth, cross-org, or externally-triggered calls (auth, `users/me`, `invitations-by-token`, the public widget, Stripe's webhook, `/admin`) break that pattern.
- Every list `GET` should support `?page=&limit=&sort=` plus resource-appropriate filters (`?status=open`) — not spelled out per line above.
- RBAC: gate member-management, billing, API keys, and webhooks to Owner/Admin; Members can generally stay read-only on conversations and analytics. That's the actual payoff of modeling `OrganizationMember` as its own table.

Roughly 150 endpoints total. Build order: Auth → Organizations → Agents → Knowledge Base → Conversations → Messages gets a working product; Tickets and Analytics come next; API keys, webhooks, billing, and admin are what turn it from a project into a product.